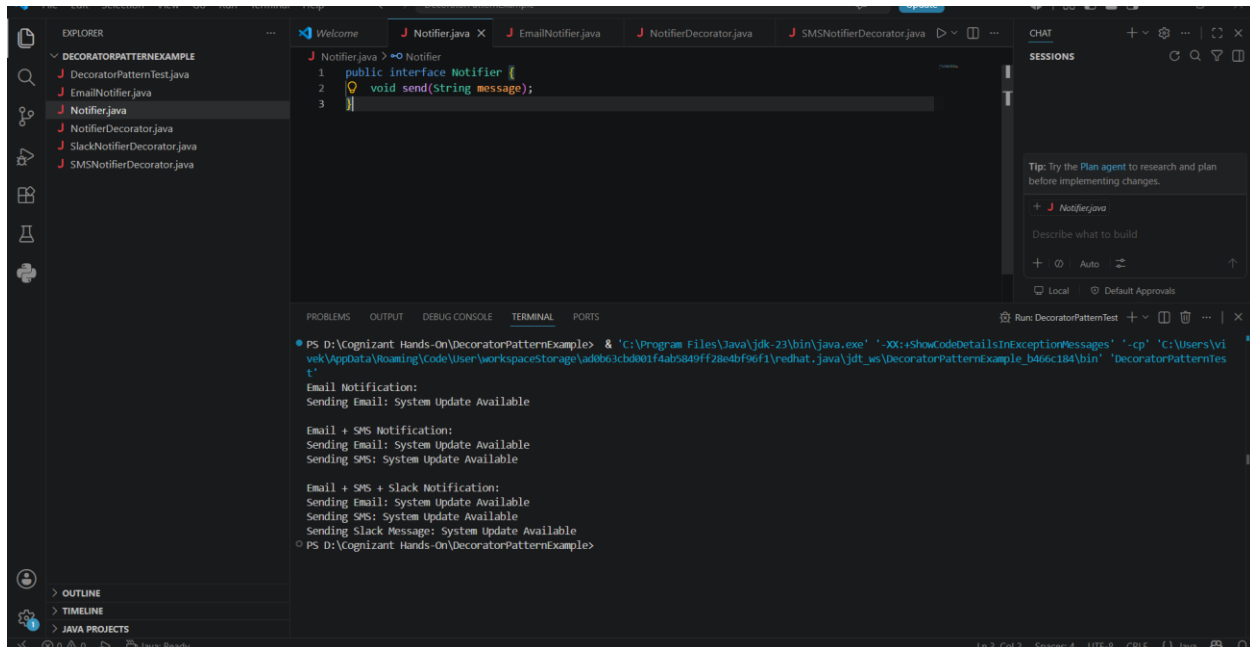


Hands-On

Exercise 5: Implementing the Decorator Pattern

As part of this module, I successfully implemented, executed, and tested the Decorator Pattern in Java by developing a project named DecoratorPatternExample. I created a Notifier interface with a send() method and implemented an EmailNotifier class as the base notification component. To add additional notification channels dynamically, I developed an abstract NotifierDecorator class and concrete decorators such as SMSNotifierDecorator and SlackNotifierDecorator. After completing the implementation, I executed the program and tested different combinations of notification channels to verify that new functionalities could be added without modifying the existing code. This hands-on exercise helped me understand how the Decorator Pattern enhances flexibility by dynamically extending the behavior of objects while maintaining a clean and maintainable code structure.

Coding:

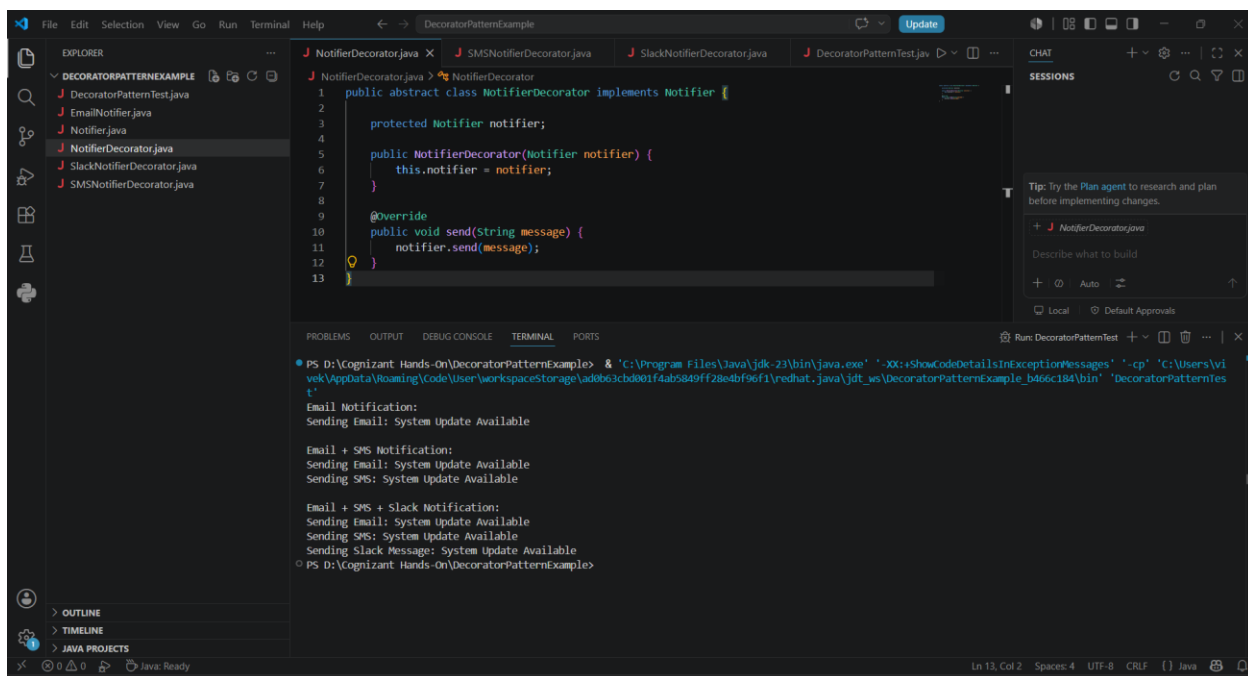
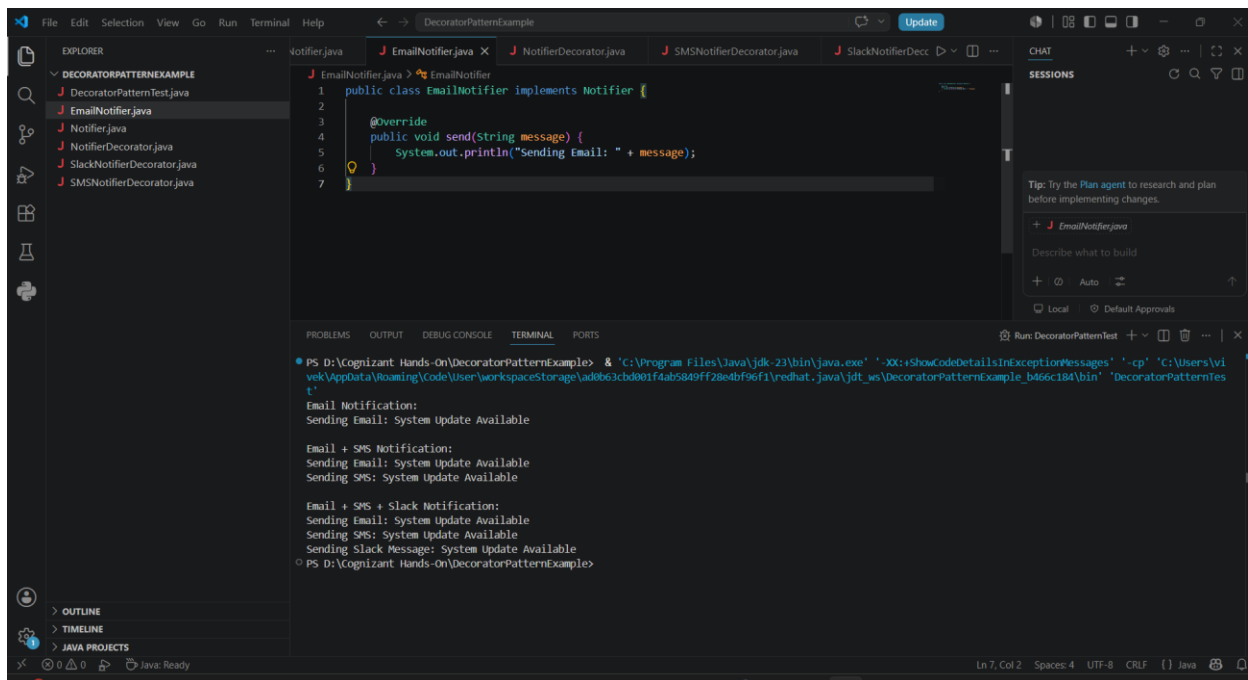


```
1 public interface Notifier {
2     void send(String message);
3 }
```

```
PS D:\Cognizant Hands-On\DecoratorPatternExample> & 'C:\Program Files\Java\jdk-23\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\viwek\AppData\Roaming\Code\User\workspaceStorage\ad863cbd001f4ab5849ff28e4bf96f1\redhat.java\jdt_ws\DecoratorPatternExample_b466c184\bin' 'DecoratorPatternTest'
Email Notification:
Sending Email: System Update Available

Email + SMS Notification:
Sending Email: System Update Available
Sending SMS: System Update Available

Email + SMS + Slack Notification:
Sending Email: System Update Available
Sending SMS: System Update Available
Sending Slack Message: System Update Available
PS D:\Cognizant Hands-On\DecoratorPatternExample>
```



The screenshot shows the VS Code editor with the `SMSNotifierDecorator.java` file open. The code defines a class that extends `NotifierDecorator` and overrides the `send` method to print an SMS notification. The terminal output shows the results of running the application, demonstrating the decorator pattern for SMS notifications.

```
public class SMSNotifierDecorator extends NotifierDecorator {  
    public SMSNotifierDecorator(Notifier notifier) {  
        super(notifier);  
    }  
  
    @Override  
    public void send(String message) {  
        super.send(message);  
        System.out.println("Sending SMS: " + message);  
    }  
}
```

Terminal Output:

```
PS D:\Cognizant Hands-On\DecoratorPatternExample> & 'C:\Program Files\Java\jdk-23\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\vi  
vek\AppData\Roaming\Code\User\workspaceStorage\ad863cbd001f4ab5849ff28e4bf96f1\redhat_java\jdt_ws\DecoratorPatternExample_b466c184\bin' 'DecoratorPatternTes  
t'  
Email Notification:  
Sending Email: System Update Available  
  
Email + SMS Notification:  
Sending Email: System Update Available  
Sending SMS: System Update Available  
  
Email + SMS + Slack Notification:  
Sending Email: System Update Available  
Sending SMS: System Update Available  
Sending Slack Message: System Update Available  
PS D:\Cognizant Hands-On\DecoratorPatternExample>
```

The screenshot shows the VS Code editor with the `SlackNotifierDecorator.java` file open. The code defines a class that extends `NotifierDecorator` and overrides the `send` method to print a Slack notification. The terminal output shows the results of running the application, demonstrating the decorator pattern for Slack notifications.

```
public class SlackNotifierDecorator extends NotifierDecorator {  
    public SlackNotifierDecorator(Notifier notifier) {  
        super(notifier);  
    }  
  
    @Override  
    public void send(String message) {  
        super.send(message);  
        System.out.println("Sending Slack Message: " + message);  
    }  
}
```

Terminal Output:

```
PS D:\Cognizant Hands-On\DecoratorPatternExample> & 'C:\Program Files\Java\jdk-23\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\vi  
vek\AppData\Roaming\Code\User\workspaceStorage\ad863cbd001f4ab5849ff28e4bf96f1\redhat_java\jdt_ws\DecoratorPatternExample_b466c184\bin' 'DecoratorPatternTes  
t'  
Email Notification:  
Sending Email: System Update Available  
  
Email + SMS Notification:  
Sending Email: System Update Available  
Sending SMS: System Update Available  
  
Email + SMS + Slack Notification:  
Sending Email: System Update Available  
Sending SMS: System Update Available  
Sending Slack Message: System Update Available  
PS D:\Cognizant Hands-On\DecoratorPatternExample>
```

Output:

